Department of Computer Science and Engineering
Indian Institute of Technology Bombay

CS 236: Operating Systems Lab Spring 2025-26 LabQuiz5 (20 marks)	A

Instructions

- No Internet. No phone. No devices. No handwritten notes. No books.
- No friends (strictly for the duration of the exam only!).
- Unethical practices will be an automatic recommendation for the FR grade.
- **Hardcoding outputs will result in a -5 mark per instance.**
- Please read the questions and submission instructions carefully.
- The labquiz consists of 3 problems. Each question is worth 10 marks.
- **Solve any 2 out of 3.**
- **Bonus of up to 5 marks for the 3rd solution.**

Submission Guidelines

- **Untar and cd into the labquiz5 directory on your Desktop. Use:**
  - ***./run.sh* to run xv6 in containerized environment**
  - ***submit-xv6* to submit (ask a TA to enter password)**
- **Ensure that your final xv6 submission compiles without any errors.**

  Submissions that do not compile will not be considered for evaluation.

1. procfs

Write an implementation of **procfs,** a new file type in the xv6 file system.
The following files are of type procfs — **procinfo**, **meminfo,** and **fsinfo**.

- procfs files are not stored on the disk and hence do not appear in the **ls** listing.
- These files have to be opened only in read-only (O_RDONLY) mode.
- Only the following functions work on the procfs files — **open**, **read**, and **close**
  All other file system calls should fail.
- The usage and semantics of open, read, and close are as follows —

  open — returns a fd and creates a corresponding struct file object. The file object may need changes to store the procfs fd type, and also information about which procfs file this struct file map to.

  read — uses fd to locate the corresponding struct file to determine fd type and the procfs file. Then invokes a kernel function to generate content described below and returns 0 if successful, else -1.
  Note read does not add content to the buffer provided as an argument, but prints information in the system call itself (directly to the console)

  close — cleans up the FS state

- Following is the information to be printed to the console when a read syscall is issued on the fd corresponding to each of the **procfs** filenames mentioned —

- **procinfo**: for each process, print its **`<pid state name>`**

- **meminfo**: print the total free physical memory available in the system (in bytes)

- **fsinfo**: for each process, print
  pid of process
  list of open FDs of the process
  the number of file structs in the file table with reference count > 1
  number of misses and hits of the buffer cache (across all file accesses)

  You may need to maintain additional variables for this purpose.
  Do not forget to use locks when necessary.
  Refer to the test cases for the required printing format.

Sample output/usage. Complete list of sample outputs - Appendix A.

```
Shell
$ ls
// ... should not list procinfo, meminfo or fsinfo

$ cat fsinfo
pid: 1
open FDs: 0 1 2
pid: 2
open FDs: 0 1 2
pid: 3
open FDs: 0 1 2 3
#files with refcnt > 1: 2
Bcache hits: 283, Bcache misses: 33

# opening procfs files not in O_RDONLY should fail
$ echo hi > procinfo
open procinfo failed
$ echo hi > meminfo
open meminfo failed
$ echo hi > fsinfo
open fsinfo failed
```

2. simply link

Update the xv6 file system to support symbolic links (symlinks). The symlink capability should be implemented via the following system call function definition —

`int symlink(char* target, char* linkname);`

The system call creates a new file named <linkname> of type SYMLINK, which is linked to an existing file **<target>**. The linking is achieved by writing the details of the **<target>** file into the datablock of the <linkname> file. The format of the details is defined as **`struct symlink`** in sysfile. c (as given in this question). Each symlink file has an inode and 1 datablock, which store the target's length and filename.

Symlink files **created** by the symlink system call can be read and written to via the normal fd-based file system-related system calls — read and write.

Assume that the target and links will be in the root directory only.
Further, two user-level programs **ln** and **file** are provided for testing and usage.

Sample usage

```
Shell
$ ls lorem.txt
lorem.txt      2 23 124

$ ln -s lorem.txt l.txt
$ ls lorem.txt l.txt
lorem.txt      2 23 124
l.txt          2 23 124
$ cat l.txt
Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor
incididunt ut labore et dolore magna aliqua.

$ file lorem.txt
lorem.txt: type=2 (file), inode refcount=1
$ file l.txt
l.txt: type=4 (symlink), inode refcount=1, target=lorem.txt

$ echo Something New! >lorem.txt
$ cat l.txt
Something New!
$ echo World Peace!! >l.txt
$ cat lorem.txt
World Peace!!
```

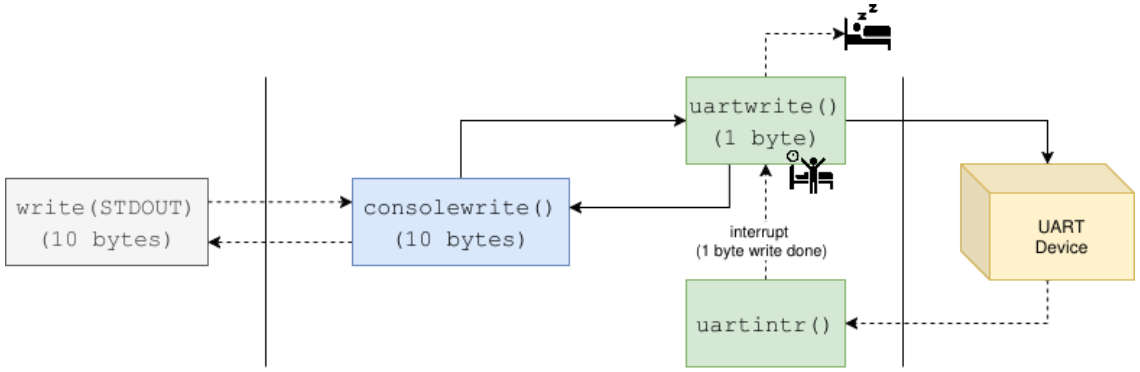
```
$ ln lorem.txt l2.txt
$ file lorem.txt
lorem.txt: type=2 (file), inode refcount=2
$ file l2.txt
l2.txt: type=2 (file), inode refcount=2
$ file l.txt
l.txt: type=4 (symlink), inode refcount=1, target=lorem.txt

$ tc_symlink

========== SYMLINK TEST SUITE ==========

[STEP 1] Creating original file 'orig.txt'...
SUCCESS: Created 'orig.txt' with content 'original content'

[STEP 2] Creating symbolic link 'link.txt' -> 'orig.txt'...
SUCCESS: Symlink created

[STEP 3] Verifying file types...
orig.txt type: T_FILE  ✓
link.txt type: T_SYMLINK  ✓

[STEP 4] Testing readlink syscall...
readlink('link.txt') = 'orig.txt'  ✓

[STEP 5] Checking inode reference counts...
orig.txt inode refcount: 1  ✓
link.txt inode refcount: 1  ✓
(Symlinks have separate inodes, don't increment target refcount)

[STEP 6] Testing write redirection through symlink...
Writing 'through symlink' to link.txt...
Reading from orig.txt to verify write went through...
Result: orig.txt contains 'through symlink'  ✓
(Write through symlink correctly redirected to target!)

[STEP 7] Testing read redirection through symlink...
Writing 'direct to orig' to orig.txt...
Reading from link.txt to verify read goes through...
Result: link.txt contains 'direct to orig'  ✓
(Read through symlink correctly resolved to target!)

========== ALL SYMLINK TESTS PASSED! ==========
```

3. No interruptions!

With the default xv6, userspace interaction with the console is driven by interrupts. Whenever a read or write operation is performed, the UART device driver relies on hardware interrupts to manage the data transfer.

As part of this question, modify the UART driver so that it does not use interrupts. Instead, all read and write operations must be handled via **polling**—a technique in which the system continuously checks the device's hardware status to determine whether it requires attention.

Task 1: Transmit (TX) Polling
When a write is issued to the console, the data follows the path shown in the figure below:



Your task is to replace the existing interrupt-driven **sleep/wakeup** semantics with a polling mechanism. The sleep/wakeup loop should be removed and replaced with the polling-based logic. This polling loop should be triggered directly by the write system call for all console write operations.

Note:
xv6 already uses polling for internal kernel prints. Whenever **`printf()`** is invoked in the kernel uses **`uartputc_sync()`** to write to the device without interrupts.

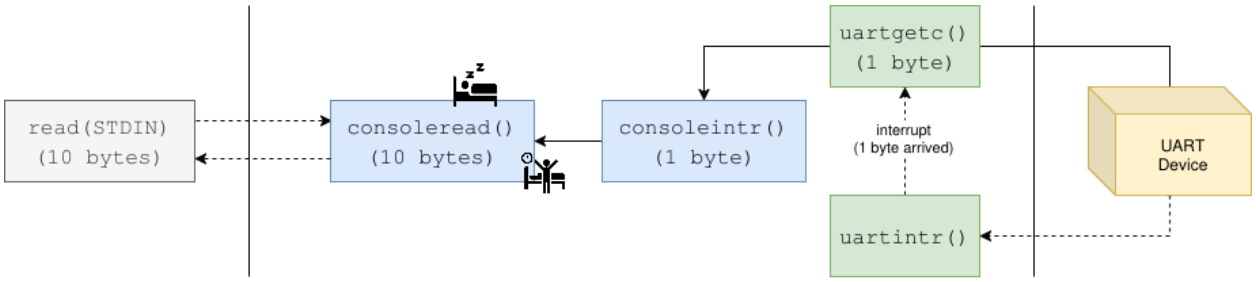
Testing:
You can verify your implementation using the `toggle` utility, or by running the test case `itests txoff`

```
C/C++
$ itests txoff
Testing busy-waiting behavior of UART interrupts.
[TX off, RX on]
Press Enter to start this setup...

[read]
```

```
Type "1234" and press Enter. Waiting up to 100 ticks...
1234
SUCCESS: read completed within timeout.

[write]
Writing "1234". Waiting up to 100 ticks...
1234
SUCCESS: write completed within timeout.
```

Task 2: Receive (RX) Polling

Similarly, when a read is issued to the console, the data follows the path shown in the figure below:



Just as with the transmit operation, you must set up RX polling logic. When a read system call is issued to the console, your implementation should continuously poll the UART hardware to check for data and deliver it to userspace.

Testing:

You can verify your implementation using the `toggle` utility, or by running the test case `itests rxoff`

```
C/C++
$ itests rxoff
Testing busy-waiting behavior of UART interrupts.

[RX off, TX on]
Press Enter to start this setup...


[read]
Type "1234" and press Enter. Waiting up to 100 ticks...
```

```
1234

SUCCESS: read completed within timeout.

[write]
Writing "1234". Waiting up to 100 ticks...
1234

SUCCESS: write completed within timeout.
```

If you successfully complete both Task 1 and Task 2, you will be able to toggle UART interrupts off entirely. You should be able to interact with xv6 normally without any functional issues, though you will notice that one CPU core will be constantly occupied by the polling loop.

```
C/C++
$ itests alloff
Testing busy-waiting behavior of UART interrupts.

[All off]
Press Enter to start this setup...


[read]
Type "1234" and press Enter. Waiting up to 100 ticks...
1234

SUCCESS: read completed within timeout.

[write]
Writing "1234". Waiting up to 100 ticks...
1234

SUCCESS: write completed within timeout.
$ toggle alloff
$ echo Hello
Hello
$
```

Appendix A

```shell
# opening and reading procfs files works
$ cat procinfo
1 sleep  init
2 sleep  sh
3 run    cat

$ cat meminfo
physical memory left: 133263360B

$ cat fsinfo
pid: 1
open FDs: 0 1 2
pid: 2
open FDs: 0 1 2
pid: 3
open FDs: 0 1 2 3
#files with refcnt > 1: 2
Bcache hits: 283, Bcache misses: 33


# opening procfs files not in O_RDONLY should fail
$ echo hi > procinfo
open procinfo failed
$ echo hi > meminfo
open meminfo failed
$ echo hi > fsinfo
open fsinfo failed


# meminfo actually works! (physical memory left reducing by almost 4MB each time)
$ cat meminfo
memory left: 133263360B

$ p1a &
$ cat meminfo
memory left: 129015808B
$ p1a &
$ cat meminfo
memory left: 124768256B
```

Shell

```
# fsinfo works! p1b opens a single file, and keeps looping indefinitely
# (pid checked via procinfo)
$ cat fsinfo
pid: 1
open FDs: 0 1 2
pid: 2
open FDs: 0 1 2
pid: 3
open FDs: 0 1 2 3
#files with refcnt > 1: 2
Bcache hits: 283, Bcache misses: 33

$ p1b &
$ cat fsinfo
pid: 1
open FDs: 0 1 2
pid: 2
open FDs: 0 1 2
pid: 6
open FDs: 0 1 2 3
pid: 5
open FDs: 0 1 2 3
#files with refcnt > 1: 3
Bcache hits: 329, Bcache misses: 37
$ cat procinfo
1 sleep  init
2 sleep  sh
7 run    cat
5 run    p1b
```

Shell

```
# fsinfo shows correct info even when p1c opens many files!
$ cat fsinfo
pid: 1
open FDs: 0 1 2
pid: 2
open FDs: 0 1 2
pid: 3
open FDs: 0 1 2 3
#files with refcnt > 1: 2
Bcache hits: 543, Bcache misses: 36
```

```
$ p1c &
$ cat fsinfo
pid: 1
open FDs: 0 1 2
pid: 2
open FDs: 0 1 2
pid: 6
open FDs: 0 1 2 3
pid: 5
open FDs: 0 1 2 3 4 5 6 7
#files with refcnt > 1: 7
Bcache hits: 602, Bcache misses: 40
$ cat procinfo
1 sleep  init
2 sleep  sh
7 run    cat
5 run    p1c
```

Shell

```
# don't modify cat!
$ p1d
1 sleep  init
2 sleep  sh
3 run    p1d
physical memory left: 133263360B
pid: 1
open FDs: 0 1 2
pid: 2
open FDs: 0 1 2
pid: 3
open FDs: 0 1 2 3
#files with refcnt > 1: 2
Bcache hits: 305, Bcache misses: 33
```

Shell

```
# opening a file (eg. README) for the first time should lead to bcache misses
$ cat fsinfo
pid: 1
open FDs: 0 1 2
pid: 2
open FDs: 0 1 2
```

```
pid: 3
open FDs: 0 1 2 3
#files with refcnt > 1: 2
Bcache hits: 283, Bcache misses: 33

$ cat README
<output redacted>
$ cat fsinfo
pid: 1
open FDs: 0 1 2
pid: 2
open FDs: 0 1 2
pid: 5
open FDs: 0 1 2 3
#files with refcnt > 1: 2
Bcache hits: 315, Bcache misses: 36

# continuing from the previous testcase, reading something alreeady in cache
doesn't lead to more bcache misses.
$ cat README
<output redacted>
$ cat fsinfo
pid: 1
open FDs: 0 1 2
pid: 2
open FDs: 0 1 2
pid: 7
open FDs: 0 1 2 3
#files with refcnt > 1: 2
Bcache hits: 350, Bcache misses: 36
```